Nyaya Sahayak: an AI legal helper for everyday problems in India

Kuku AI Generalist Assignment, Problem 2 | **PENDING** [Your name], **PENDING** [Roll no.], **PENDING** [Branch] | Prototype: **PENDING** [GitHub / hosted link]

1. Why I picked this problem

People rarely get legal help for small disputes: a deposit the landlord keeps, a refund that never arrives, salary that stops. A lawyer feels too costly for a Rs 5,000 problem, and the paperwork is scary, so people give up. I also picked it because it tests honesty under pressure: language models confidently invent laws, and a legal tool that does that is worse than no tool. My users are ordinary people in India on a phone or laptop, comfortable in English or Hindi, with a dispute of roughly Rs 5,000 to a few lakh and no lawyer.

2. Scope and assumptions

- **Three problem types only:** rent deposits, consumer refunds/complaints, unpaid or delayed salary. Anything else (criminal, family, property title) gets a plain "outside scope, see an advocate" message.
- **Tenant law is state-specific** and I did not load any state rent acts, so deposit guidance is deliberately general and tells the user to check their state.
- **Law as of November 2026.** For example the Payment of Wages Act, 1936 was repealed on 21 Nov 2025 and replaced by the Code on Wages, 2019; the knowledge base reflects that.
- **"Substantial amount" = Rs 2 lakh** (a setting) triggers an advocate warning. No user accounts; a case lives in a random id, kept 30 days and deletable. PDF export uses the browser's print dialog.

3. How it works

Each message goes through these steps. The model does language work; ordinary code makes the decisions that must be reliable. **(1)** Redact card/Aadhaar-like numbers and OTPs. **(2)** Code scans for danger signs in English and Hindi: court notice or summons, police/criminal, ongoing case, family/property, safety emergencies. **(3)** The model extracts facts into fixed slots (location, dates, amount, proof) using only what the user said. **(4)** Code finds missing slots and asks at most three questions, with no legal conclusion yet (it moves on after three rounds or when the user taps "Get guidance now"). **(5)** Code retrieves matching entries from a small curated knowledge base. **(6)** The model writes guidance or a draft letter, allowed to cite knowledge-base ids only. **(7)** Code validates citations and adds the advocate warning. Output is a case file (facts told vs missing vs AI's readings), guidance with sources, and an editable letter (.txt or PDF), in English or Hindi, with voice input where the browser supports it.

4. Technology and what AI vs code vs a human does

Python standard library server, SQLite, plain HTML/JS: zero installs, easy for anyone to run. The model sits behind a three-provider adapter (Gemini, any OpenAI-compatible API, Anthropic), so swapping models is two lines in `.env`. **AI:** understanding messy stories, plain-language explanation, drafting, Hindi. **Code:** slot logic, retrieval, citation checks, safety rules, escalation, redaction. **Human:** an advocate must review the knowledge base and every draft before it is sent or filed; the app says so on every draft.

5. Legal accuracy and hallucination prevention

- The model cannot introduce a citation. It may only name knowledge-base ids; the act, section, official link and check date shown to the user are built by code from the knowledge base.
- A section number is shown only when that entry's section was actually checked. Unchecked ones are withheld from the model entirely. The system never cites court judgments.
- Any leftover section number or case citation in the model's text that no verified entry backs is stripped and reported to the user. If the user quotes a section, code checks it and says "I could not verify that" instead of describing it.
- **Honest status:** the knowledge base has 18 entries, each labelled verified / secondary-verified / provisional / general principle. Only Consumer Protection Act s.69 was read as bare-act text, and on a third-party site, not India Code. The rest rest on law-firm and legal-news sources or general principles. The official links must still be opened and an advocate must review the entries before real use.

6. Testing and results

Method. Seven realistic scenarios run through the real pipeline and a real model (**PENDING [exact provider and model name used for the test run, e.g. provider / model]**, run **PENDING** [date you ran tests/run_scenarios.py]): the six required cases plus a Hindi one. Raw outputs are saved unedited in `raw_outputs/`. Automated checks are only a first filter; I read each output and scored it on eight points (legal accuracy, citation reliability, follow-up questions, document quality, language, hallucination prevention, user experience, correct escalation), checking each cited provision against its official link.

Scenario	What I checked	What actually happened
T1 Rental deposit	Asks city/state, agreement, dates; guidance; demand notice with placeholders	**PENDING**
T2 Online refund	Asks purchase date, seller, proof; consumer forums; refund request draft	**PENDING**
T3 Unpaid salary	Asks employment type, dates, salary terms; cites Code on Wages, not repealed Act; demand letter	**PENDING**
T4 Vague complaint	Only questions, no legal conclusion	**PENDING**

Scenario	What I checked	What actually happened
T5 Invented section	Says it cannot verify; does not invent what the section says	**PENDING**
T6 Court notice	Flags urgency, advises advocate, invents no deadline	**PENDING**
T7 Hindi deposit (bonus)	Hindi replies and letter keep dates, amounts and names exact	**PENDING**

Checks already done on the code itself: 16 offline unit tests of the safeguards (citation scrubber, safety rules, redaction, knowledge-base integrity) pass. A plumbing run with a stand-in model caught two of my own bugs before any real run: a real answer of "none" was treated as missing information, and the scrubber left sentence fragments around removed citations. Both are fixed. These tests show the code works; they say nothing about the model's answer quality, which is what the table above is for.

Biggest failure I saw: **PENDING**

7. Limitations and failure cases

- The validator catches invented section numbers and case citations by pattern. It cannot catch a wrong claim written in ordinary words, or a right number attached to the wrong act when both appear close together.
- Safety rules are keyword-based and can miss unusual wording or over-trigger (they lean towards sending people to an advocate). The model can still misclassify a case.
- Hindi quality depends on the model and has not been checked by a Hindi-speaking lawyer; citation text is English. No state rent law, so tenant advice stays general.
- It relies on free model tiers. During setup I hit a retired default model name (HTTP 404), busy servers (503) and rate limits (429); the app now retries with waits, and the model is set in `.env`. Not deployed yet; single-browser sessions only.

8. Scaling

A case costs about two model calls per message plus one per drafted document, and retrieval is free, so a cheaper fast model can do extraction while a stronger one writes guidance. The real cost of scale is keeping the law correct: laws change (the wage-law repeal is an example), so each entry has a check date and the next step is a scheduled re-verification plus advocate sign-off. Adding a category is a new slot list, a few knowledge-base entries and test cases; adding a state means advocate-reviewed state rent-law entries. Next steps: retrieval over official PDFs with passage-level citations, an evaluation set of 100+ real cases, proper accounts and encrypted storage.